

Assignment 3 – LLRB Trees

Task 2: When examining deletion in LLRB trees, it's important to think about how LLRB trees correlate with 2-3 trees. The solution Robert Sedgewick presents in the paper below is reliant on adjusting the tree slightly while moving down and searching for the node and ensuring that the search doesn't end on a 2 node, allowing us to simply delete the key from the node in the 2-3 tree representation.

Sedgewick's solution also relies on a `fixup()` function that moves back up the tree to clean up any right red links that are left because of the rotate operations on the way down the tree. Since delete is called recursively, `fixup()` is called for any instance where we might've left a right red link in the tree.

Sources:

<https://sedgewick.io/wp-content/themes/sedgewick/papers/2008LLRB.pdf>

Task 3: Left leaning red black trees seem simple enough in that their only new restriction compared to red black trees are that if a node only has one red child, it can only be left children of their parent. This property means that they're equivalent to 2-3-4 trees, while changing the property so that red nodes can *only* be left children creates an equivalence with 2-3 trees (since you can't have the equivalent to 4 nodes).

Now onto my analysis of the article. Overall, it claims that conventional RB are more simple to implement and more efficient when compared to LLRB trees, and their widespread implementation makes it easier to use someone else's implementation. The author claims that "The simplest code is the code you didn't write. So use someone else's red-black tree (or other search tree structure) if you can". The article quotes Jason Evans' claim that left leaning red black trees have an equally complex implementation compared to standard red-black trees. I can also slightly agree with their claim on how Sedgewick's paper doesn't highlight the difference between 2-3 and 2-3-4 trees compared to LLRB trees as much as it should in order to express how its delete function works. The author also cites data from an experiment benchmarking the value of parent pointers, finding that rb trees with parent are very slightly slower, but still contesting that they allow the finding of

successor/predecessor with amortized $O(1)$ time. Ultimately, the article shows that LLRB trees are more complex and less efficient compared to traditional red black trees.